For this homework, you will need to understand the notes on Recurrent Neural Networks.

1) Where is the gradient?

In backpropagation through time, we have a nice recursive equation for δ_i^{st-1}, which is the degree to which unit i of the state at time $t-1$ was responsible for the losses from time t through n (the end of the sequence):

$$\delta_i^{st-1} = \sum_{j=1}^{m} \frac{\partial s_{t;j}}{\partial s_{t-1;i}} \left[\frac{\partial}{\partial s_{t;j}} \text{Loss}(y_t, p_t) + \delta_j^{st} \right]$$

For concreteness, consider a sequence of length $n = 5$ and assume that s has a single dimension, so we can omit that subscript.

1A) We are interested in the impact of the state at time 1 on the error at time 5. Assume that the losses for all times $t < 5$ are 0.

Which of the following expressions are correct formulas for that quantity?

- [x] $$\delta^{s1} = \frac{\partial s2}{\partial s1}\delta^{s2}$$
- [x] $$\delta^{s1} = \frac{\partial s2}{\partial s1}\frac{\partial s3}{\partial s2}\delta^{s3}$$
- [ ] $$\delta^{s1} = \left(\sum_{t=2}^{5}\frac{\partial st}{\partial st-1}\right)\left(\frac{\partial}{\partial s5}\text{Loss}(y5, p5)\right)$$
- [x] $$\delta^{s1} = \left(\prod_{t=2}^{5}\frac{\partial st}{\partial st-1}\right)\left(\frac{\partial}{\partial s5}\text{Loss}(y5, p5)\right)$$

Submit | View Answer | Ask for Help **100.00%**

You have infinitely many submissions remaining.

1B) If the network were actually 20 layers deep, $\frac{\partial st}{\partial st-1} = 0.1$ for all t, and $\delta^{s20} = 1$, what is δ^{s1}?

Enter a number: 1e-19

Submit View Answer Ask for Help 100.00%

You have infinitely many submissions remaining.

1C) Is this gradient exploding or vanishing?

Pick one: Vanishing

Submit View Answer Ask for Help 100.00%

You have infinitely many submissions remaining.

2) RNN

Consider a simple recurrent neural network model given by

$$s_t = \text{sign}_0(W^{ss} s_{t-1} + W^{sx} x_t), \quad t = 1, 2, \ldots$$

where we have omitted any offset parameters (set biases to zero) and the non-linear activation function, applied element-wise, is a sign function that also returns zero: $\text{sign}_0(z) = 1$ if $z > 0$, $\text{sign}_0(0) = 0$, and -1 otherwise. In our problem here, $s \in \mathbb{R}^2$ and $x \in \mathbb{R}$. The parameters are set (not learned) as follows:

$$W^{ss} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}, \quad W^{sx} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad s_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

We will use the RNN model to translate variable length input sequences to corresponding state vectors. In other words, a sequence x_1, x_2 will be represented by the resulting s_2; similarly, x_1, x_2, x_3 will be represented by s_3, and so on.

Once each sequence has an associated state vector, we can use them, e.g., to train a linear classifier. To this end, our three training examples are binary sequences:

$$x^{(1)} = (x_1^{(1)}, x_2^{(1)}) = (1, 0)$$

$$x^{(2)} = (x_1^{(2)}, x_2^{(2)}, x_3^{(2)}) = (0, 0, 1)$$

$$x^{(3)} = (x_1^{(3)}) = (0)$$

Note that the input sequences do not have to have the same length! (This is an advantage of RNNs).

2A) What is the sequence of first two state vectors s_1 and s_2 resulting from feeding the RNN model with the input sequence $x^{(1)}$?

Enter a list of two numbers corresponding to s_1:

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

Enter a list of two numbers corresponding to s_2:

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

2B) Map all the sequences to their corresponding state vector representations (that is, the final state vector that results from feeding the entire sequence into the model):

Enter a list of two numbers for the final state vector given input $x^{(1)}$:

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

Enter a list of two numbers for the final state vector given input $x^{(2)}$:

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

Enter a list of two numbers for the final state vector given input $x^{(3)}$:

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

2C) Suppose we train a linear classifier that operates on the state vector representations of these sequences $x^{(1)}$, $x^{(2)}$, and $x^{(3)}$ (run through our RNN). Could the linear classifier separate these three examples for all possible labelings of positive and negative examples?

Could the linear classifier separate these three examples regardless of how they were labeled? --

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

2D) We can think of the RNN model as giving rise to an equivalent feed-forward neural network for any input sequence $(x_1, x_2, \ldots, x_n)$ run through it. Choose True or False for each statement based on whether it is correct for this feed-forward interpretation:

The parameters of each layer are different: --

Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

Each input x_i is fed into a different hidden layer: --

Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

The number of layers is proportional to the length of the input sequence: --

Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

Each hidden layer would have two units in case of our RNN above: --

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

3) State

We consider a language model based on sequences of characters. We construct a classifier on the state

$$s_t = \tanh\left(\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix} s_{t-1} + W^{s,x} x_t\right)$$

such that positive classification is decided if $[1,1] \cdot s_T > 0$ and negative classification is if decided $[1,1] \cdot s_T < 0$ (for s_T at the end of the word).

Find W^{sx} such that 'draw' and 'saw' result in positive classification and 'paw' and 'raw' result in negative classification. Assume $s_0 = [0,0]$.

Assume our alphabet only has the letters 'a', 'd', 'p', 'r', 's', and 'w' and that when we make a one-hot encoding, we do it in that order.

3A) What is the shape of W^{sx}?

Enter a Python list of numbers corresponding to the shape of the matrix:

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

3B) Provide a W^{sx}.

Enter the matrix as a list of lists; one list for each row of the matrix:

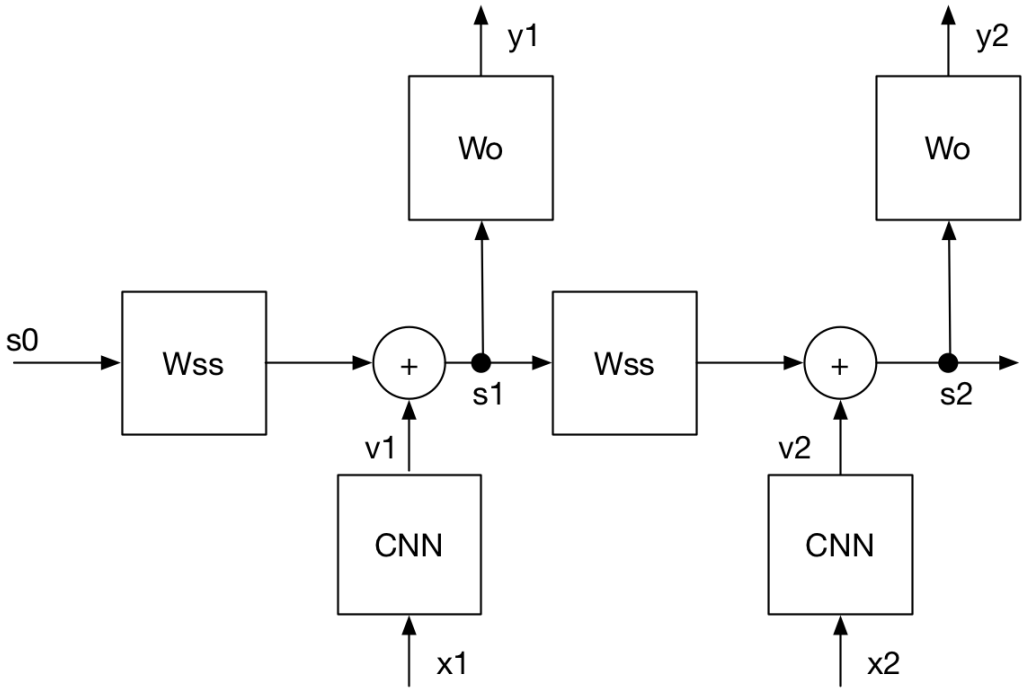
Submit View Answer Ask for Help

You have infinitely many submissions remaining.

4) A sequence of cats

Imagine that we have several training sequences. Sequence j consists of n_j images, $x_1^{(j)}, \ldots, x_{n_j}^{(j)}$, and a corresponding sequence of numbers $y_1^{(j)}, \ldots, y_{n_j}^{(j)}$ where $y_i^{(j)}$ is intended to indicate the number of images, among $x_1^{(j)}, \ldots, x_i^{(j)}$ that satisfy some desired criterion (e.g., they are images of cats).

We will use the architecture below (note that the outputs in this diagram are labelled as y_t, which are the desired outputs, the actual outputs are p_t):



In place of W^{sx} in the usual RNN, we will put a more complex, convolutional neural network CNN; but it still takes an input (in this case, an image) and returns a scalar value as output (labeled v_i in the figure).

Assume:

- The loss function for prediction on a sequence of length T is $L(p, y) = \frac{1}{2T} \sum_t (pt - yt)^2$,
- The output unit on the CNN is a sigmoid,
- W^{ss} is a scalar (assume it is 1 when numerical results are required),
- W^{o} is a scalar (assume it is 1 when numerical results are required), and
- f_1 and f_2 are linear.

Assume that y is the sequence of *true, desired outputs* and that p is the sequence of actual outputs from the network. **Note that we are looking at a sequence of two images.**

4A) Since we have replaced W^{sx} with a CNN, in order to perform gradient descent, we must calculate the loss with respect to $v1$, $v2$, the outputs of the CNN. What is $\frac{dL}{dv2}$? Provide your answer as an expression in terms of: `y_1`, `y_2`, `p_1`, `p_2`, `Wss`, `Wo`.

dL/dv2=

Check Syntax | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

4B) What is $\frac{dL}{dv1}$? Provide your answer as an expression in terms of: `y_1`, `y_2`, `p_1`, `p_2`, `Wss`, `Wo`.

dL/dv1=

Check Syntax | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

4C) Assume there were three correctly-counted cat pictures before now, and so $s_0 = 3$, the desired output values $y = (3, 4)$, and the current predicted values $p = (4, 4)$. Which images are being misclassified by the CNN?

Choose one: --

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

4D) In this case, what is $\frac{dL}{dv2}$?

Enter a number:

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

4E) What is $\frac{dL}{dv1}$? Recall that $W_{ss} = 1$ and $W_o = 1$. Look at the explanation of the answer to see a simulation of running gradient descent.

Enter a number:

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

4F) Assume $s_0 = 3$ is correct, $y = (3, 4)$, $p = (4, 5)$. Which images are being misclassified by the CNN?

Choose one: --

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

4G) In this case, what is $\frac{dL}{dv2}$?

Enter a number:

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

4H) What is $\frac{dL}{dv1}$? Look at the explanation of the answer to see a simulation of running gradient descent.

Enter a number:

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

4I) Assume $s_0 = 3$, $y = (3, 4)$, $p = (3, 3)$. Which images are being misclassified by the CNN?

Choose one: --

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

4J) In this case, what is $\frac{dL}{dv2}$?

Enter a number:

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

4K) What is $\frac{dL}{dv1}$? Look at the explanation of the answer to see a simulation of running gradient descent (and the code used to generate the simulations).

Enter a number:

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

Note: You can see that this is learning to classify the second image correctly (that is, get $v2$ correct) and then classify

the first image in the sequence correctly. If it does manage to get the second one right, then it has a good error signal for the first.

This approach is less efficient and direct compared to just getting explicitly correct labels for each image. But the point of this exercise is to hopefully give you some insight into how backpropagation through time works with recurrent neural networks and how the errors from later timesteps are used for weight updates.

5) Backprop Through Time (BPTT) in formulas

Please watch the Week 11 lecture video before doing this problem.

We will build on the notation and derivations in Chapter 12 of the class notes. The notes write out expressions in scalar form (to update individual components of the weight matrices); we will be using vector/matrix notation that matches the numpy implementation.

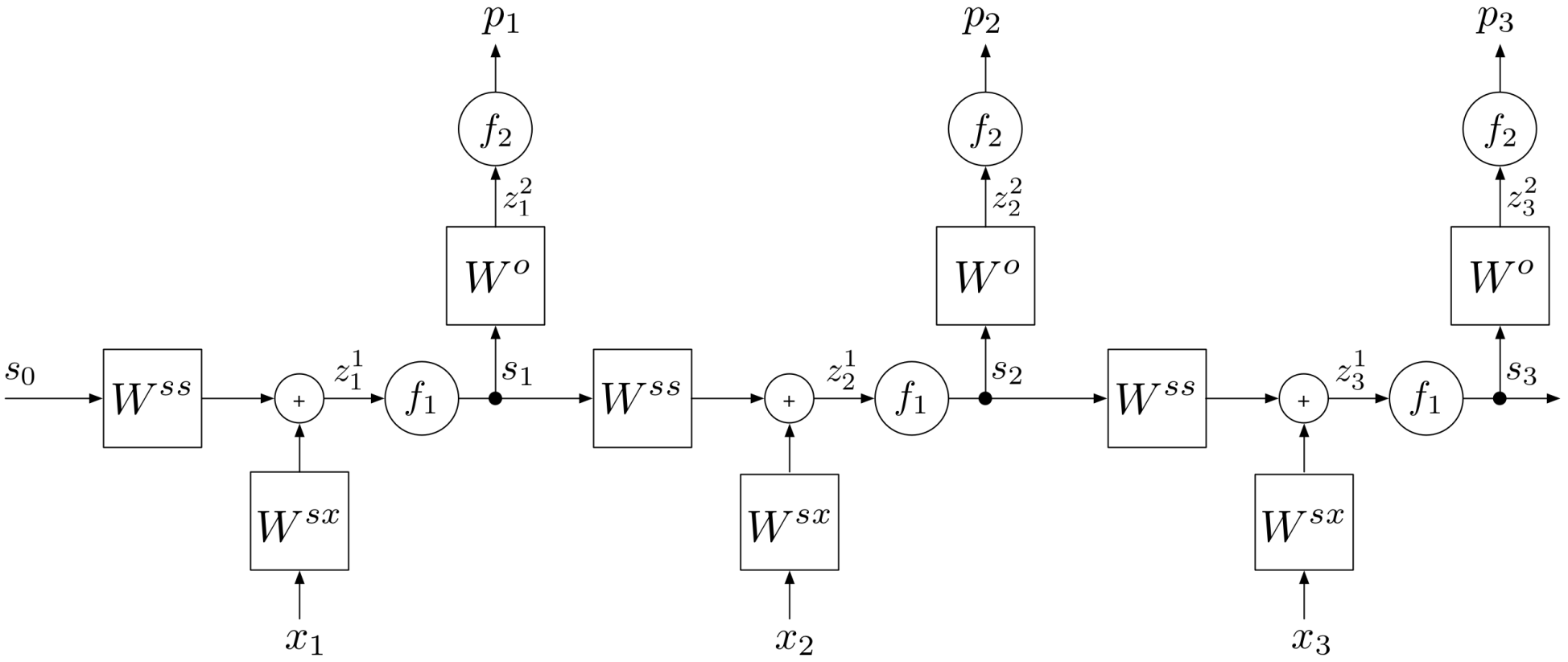
The backpropagation algorithm (as usual) will consist of:

- A forward pass that will compute the values of the quantities indicated on the "wires" of the diagram below: s_t, z_t^1, z_t^2, p_t.
- A backward pass that will compute the gradients of the relevant qunatities with respect to the weights in the matrices.

Recall that the forward pass computes:

$$
\begin{aligned}
z_t^1 &= W^{ss} s_{t-1} + W^{sx} x_t + W_0^{ss} \\
s_t &= f_1(z_t^1) \\
z_t^2 &= W^{o} s_t + W_0^{o} \\
p_t &= f_2(z_t^2)
\end{aligned}
$$

The image below shows the operation of the (forward) RNN unrolled over three time steps. We will use this to help us derive the basic equations we will need to implement the backward pass in RNNs.



Enter your answer to each of the following questions as a Python expression. You can use `transpose(x)` for transpose of an array, and `x@y` to indicate a matrix product of two arrays. Remember that `x*y` denotes component-wise multiplication.

In any of these questions you can always use: `Wss`, `Wsx`, `Wo`, `Wo_0`, `Wss_0`, `st`, `xt`, `z1t`, `z2t`. Additional allowed inputs will be described with each question.

5.1) Delta Recursion

In the backward pass, as we did for feedforward networks, we are looking for quantities that enable us to define a recursion, where we define the gradient at time (formerly layer) $t-1$ given the gradient at time (formerly layer) t. The crucial quantity for BPTT is:

$$\delta^{st} = \frac{\partial}{\partial st} \sum_{l=t+1}^{n} \text{Loss}(yl, pl)$$

This is the derivative with respect to the state st at time t, of the future losses (after time t). Note that we will refer to the loss at time t as `Lt` and the future losses (after time t) as `Ft`. Therefore, we will refer to δ^{st} as `dFtdst`. Note that $\text{Loss}(yt, pt) + \sum_{l=t+1}^{n} \text{Loss}(yl, pl)$ is the future loss for time $t-1$, which we will call `Ftm1`.

We are interested in computing δ^{st-1} given δ^{st}. We can see from Eq(8) - Eq(11) in the notes, that:

$$\delta^{st-1} = \frac{\partial st}{\partial st-1} \left[\frac{\partial}{\partial st} \text{Loss}(yt, pt) + \delta^{st} \right]$$

Assume we have $\frac{\partial \text{Loss}}{\partial z_t^2}$ available (called `dLtdz2t`); recall from Week 5 that we have a nice simple expression $(p_t - y_t)$ for this quantity for the common loss functions.

5.1A) Write an expression for $\frac{\partial}{\partial s_t}\text{Loss}(y_t, p_t)$ (called `dLtdst`).

dLtdst=

Check Syntax | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

5.1B) Assume we have `dFtdst` (defined as above) and `dLtdst`. What is $\frac{\partial}{\partial s_t}\left[\text{Loss}(y_t, p_t) + \sum_{l=t+1}^{n}\text{Loss}(y_l, p_l)\right]$ (called `dFtm1dst`)?

dFtm1dst=

Check Syntax | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

5.1C) Assume we have `dFtm1dst` (defined as above) and we have been given $\frac{df_1}{dz_t^1}$ (called `dfdz1`). What is $\frac{\partial_1}{\partial z_t^1}\left[\text{Loss}(y_t, p_t) + \sum_{l=t+1}^{n}\text{Loss}(y_l, p_l)\right]$ (called `dFtm1dz1t`)?

dFtm1dz1t=

Check Syntax | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

5.1D) Assume we have `dFtm1dz1t` (defined as above). Whats is δ^{st-1} (called `dFtm1dstm1`).

dFtm1dstm1=

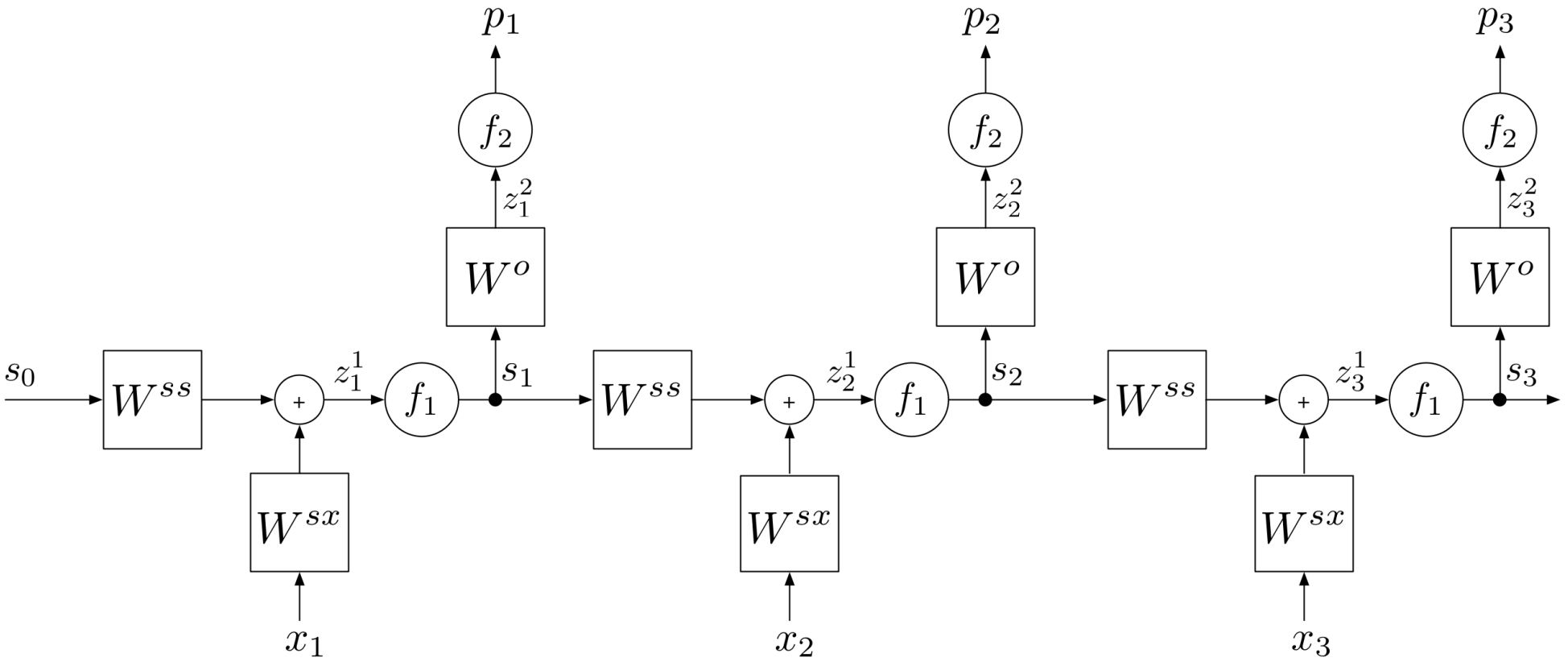
Check Syntax | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

5.2) Weight gradients

Assume we have the quantities defined in the previous section: `dFtm1dst`, `dFtm1dz1t`, `dLtdz2t`, and also s_t (called `st`), s_{t-1} (called `stm1`) and x_t (called `xt`).

This diagram is helpful to refer back to:



5.2A) What is $\frac{d}{dW^o}[\text{Loss}(y_t, p_t)]$?

dLtdWo=

Check Syntax | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

5.2B) What is $\frac{d}{dW_0^o}[\text{Loss}(y_t, p_t)]$?

dLtdWo0=

Check Syntax | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

5.2C) What is $\frac{d}{dW^{ss}}[\sum_{l=t}^{n} \text{Loss}(yl, pl)]$?

dFtm1dWss=

Check Syntax | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

5.2D) What is $\frac{d}{dW_0^{ss}}[\sum_{l=t}^{n} \text{Loss}(yl, pl)]$?

dFtm1dWss0=

Check Syntax | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

5.2E) What is $\frac{d}{dW^{sx}}[\sum_{l=t}^{n} \text{Loss}(yl, pl)]$?

dFtm1dWsx=

Check Syntax | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

6) BPTT in code

Complete the following implementation of BPTT. This is a method in an RNN class.

The code for this problem is also available in a colab notebook here.

The instance (`self`) has attributes that define the RNN, in particular:

- The weight matrices and offsets: `self.Wss`, `self.Wsx`, `self.Wo`, `self.Wss0` and `self.Wo0`.
- The activation functions and their derivatives: `self.f1`, `self.df1`, `self.f2`, `self.df2`
- The loss function and the derivative of the combined loss and final activation (as we did for feedforward networks): `self.loss_fn` and `self.dloss_f2`.
- The dimensions of states (hidden), inputs and outputs: `self.hidden_dim`, `self.input_dim`, `self.output_dim`.
- The initial state and, the current state: `self.init_state` and `self.hidden_state`
- The step size for gradient descent: `self.step_size`

```
1 # Back propgation through time
```

```
# xs is matrix of inputs: l by k
# dLdz2 is matrix of output errors:  1 by k
# states is matrix of state values: m by k
def bptt(self, xs, dLtdz2, states):
    dWsx = np.zeros_like(self.Wsx)
    dWss = np.zeros_like(self.Wss)
    dWo = np.zeros_like(self.Wo)
    dWss0 = np.zeros_like(self.Wss0)
    dWo0 = np.zeros_like(self.Wo0)
    # Derivative of future loss (from t+1 forward) wrt state at time t
    # initially 0;  will pass "back" through iterations
    dFtdst = np.zeros((self.hidden_dim, 1))
    k = xs.shape[1]
    # Technically we are considering time steps 1..k, but we need
    # to index into our xs and states with indices 0..k-1
    for t in range(k-1, -1, -1):
        # Get relevant quantities
        xt = xs[:, t:t+1]
        st = states[:, t:t+1]
        stm1 = states[:, t-1:t] if t-1 >= 0 else self.init_state
        dLtdz2t = dLtdz2[:, t:t+1]
        # Compute gradients step by step
        # ==> Use self.df1(st) to get dfdz1;
        # ==> Use self.Wo, self.Wss, etc. for weight matrices
        # derivative of loss at time t wrt state at time t
```

```
        dLtdst = None          # Your code
        # derivatives of loss from t forward
        dFtm1dst = None              # Your code
        dFtm1dz1t = None             # Your code
        dFtm1dstm1 = None            # Your code
        # gradients wrt weights
        dLtdWo = None                # Your code
        dLtdWo0 = None               # Your code
        dFtm1dWss = None             # Your code
        dFtm1dWss0 = None            # Your code
        dFtm1dWsx = None             # Your code
```

Run Code | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.